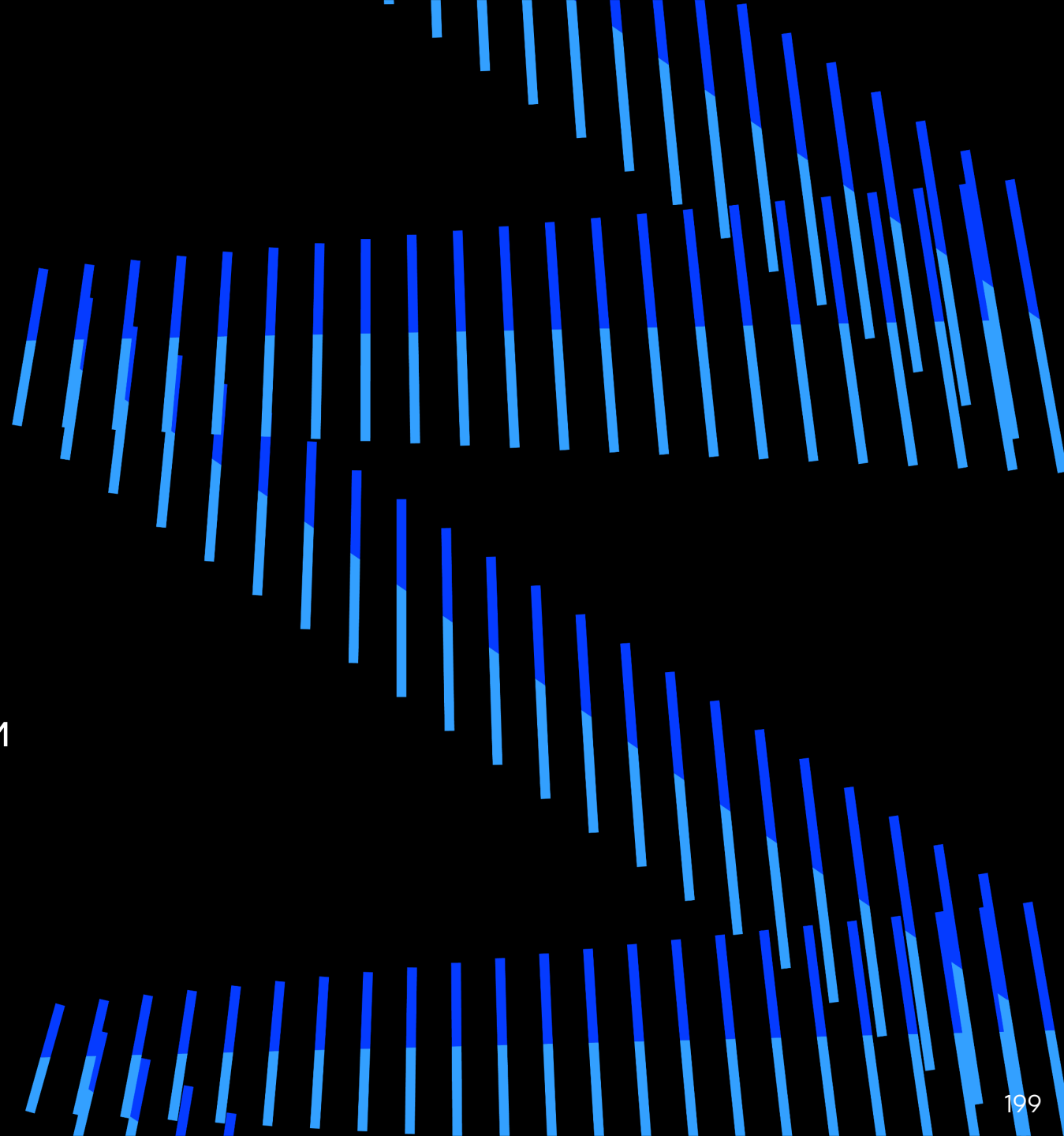


3. Управление экспериментами

План

1. Первичная настройка пайплайна: структура проекта и инструментарий
2. Первые эксперименты
3. Итеративное улучшение

Структура проекта

```
.
├── config
├── data
├── logs
├── notebooks
├── src
├── scripts
├── pyproject.toml
└── README.md
```

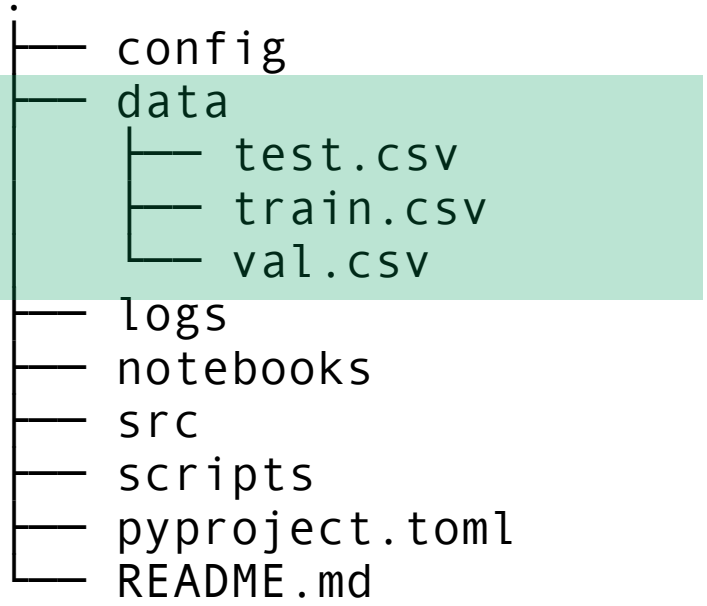
Структура проекта

```
.
├── config
│   ├── inference.yaml
│   └── train.yaml
├── data
├── logs
├── notebooks
├── src
├── scripts
├── pyproject.toml
└── README.md
```

Конфигурация всех компонентов для запуска обучения и инференса:

- исходные данные и их преобразования
- гиперпараметры: модель, оптимизатор, цикл обучения
- вспомогательные компоненты: метрики, графики, логирование

Структура проекта



Данные, используемые для обучения и тестирования

В идеале должны версионироваться вместе с кодовой базой

Структура проекта

```
.
├── config
├── data
├── logs
│   └── runs
│       ├── 2025-04-26-11:16:35
│       │   ├── checkpoints
│       │   │   └── epoch_002.ckpt
│       │   ├── tensorboard
│       │   │   ├── events.out.tfevents
│       │   │   └── hparams.yaml
│       │   └── train.log
├── notebooks
├── src
├── scripts
├── pyproject.toml
└── README.md
```

Логи экспериментов для каждого запуска:

- метадата: когда запущен, с какой версией кода, краткое описание, теги
- конфигурация
- чекпоинты
- метрики
- консольный вывод

Структура проекта

```
.
├── config
├── data
├── logs
├── notebooks
│   └── dataset.ipynb
├── src
├── scripts
├── pyproject.toml
└── README.md
```

Ноутбуки для демонтрации отдельных компонентов:

- анализ данных
- визуализация предсказаний

Удобны для отладки и быстрой проверки гипотез

Структура проекта

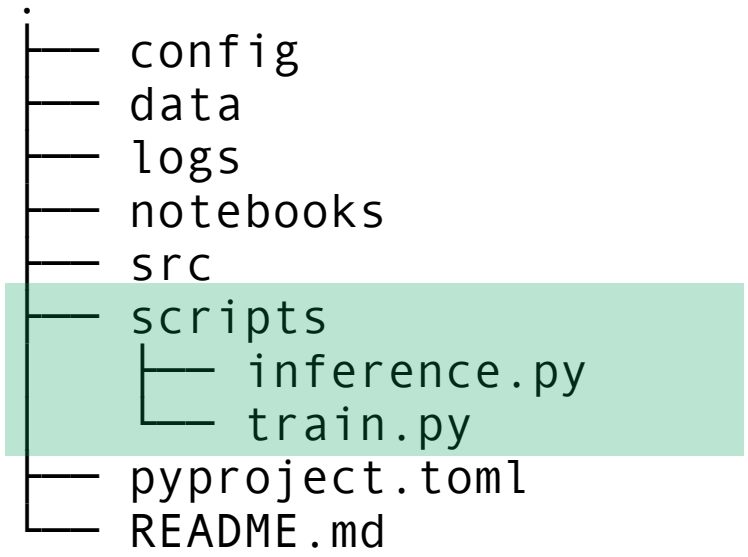
```
.
├── config
├── data
├── logs
├── notebooks
├── src
│   ├── __init__.py
│   ├── data
│   │   ├── __init__.py
│   │   ├── datamodule.py
│   │   └── dataset.py
│   ├── model
│   │   ├── __init__.py
│   │   ├── lightning.py
│   │   └── modules.py
│   └── utils
│       ├── __init__.py
│       ├── metrics.py
│       └── training.py
├── scripts
├── pyproject.toml
└── README.md
```

Код для отдельных компонентов пайплайна обучения:

- работа с данными
- архитектуры моделей
- цикл обучения
- функции ошибок
- метрики
- callbacks

Вместо src (либо внутри src) — название вашего проекта

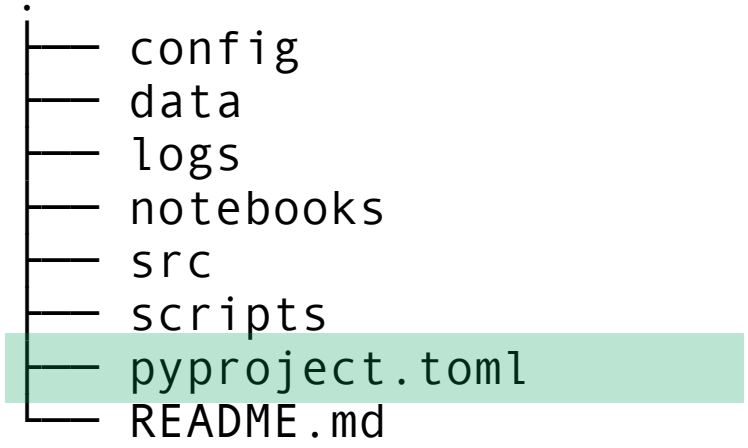
Структура проекта



Скрипты для запуска обучения и инференса

При удачной конфигурации мало варьируются от проекта к проекту

Структура проекта



Настройки python проекта:

- зависимости (пользуйтесь uv / poetry вместо pip)
- настройки проверки кода и тестирования
  - форматирование: isort, black, ruff
  - линтер: flake8, ruff
  - type checker: mypy, pyright, ty
  - testing: pytest

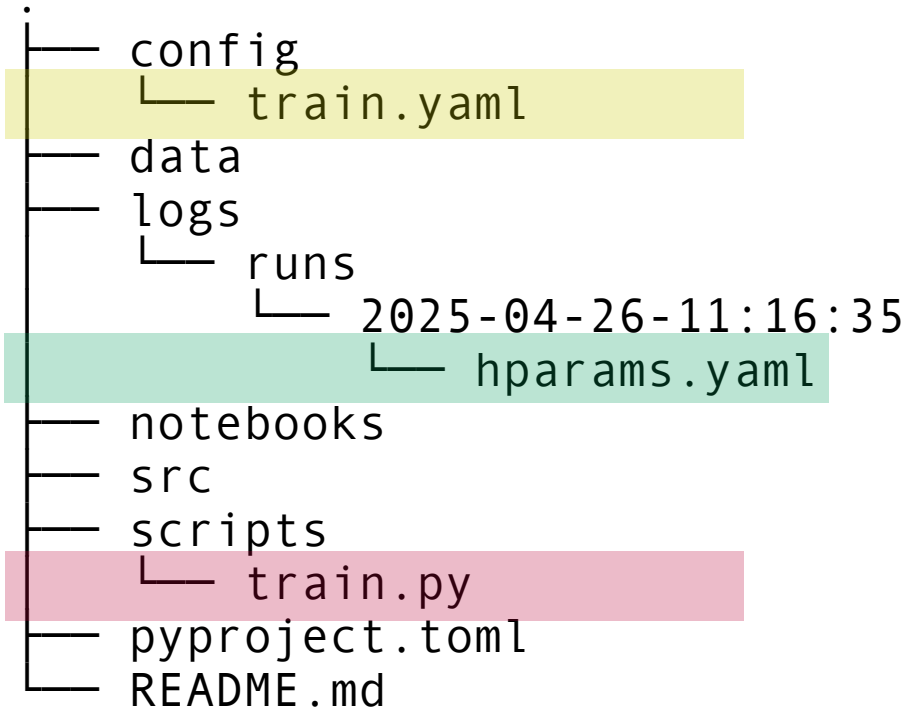
Структура проекта

```
.
├── config
├── data
├── logs
├── notebooks
├── src
├── scripts
├── pyproject.toml
└── README.md
```

Описание проекта:

- в чём задача
- основная идея решения
- результаты
- как запустить инференс
- как воспроизвести обучение

Управление конфигурацией



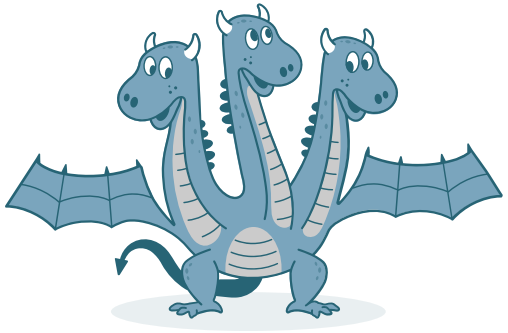
Скрипт для запуска обучения:

- берёт конфигурацию всех компонентов
- даёт возможность переопределить отдельные параметры из командной строки
- сохраняет новую конфигурацию в логи запуска и стартует обучение

Hydra

```
.
├── config
│   └── train.yaml
└── train.py
```

```yaml
seed: 42
trainer:
  accelerator: cpu
  max_epochs: 10
```



https://hydra.cc/

[1]

```python
import hydra
from omegaconf import DictConfig, OmegaConf


@hydra.main(version_base=None, config_path="config", config_name="train")
def train(cfg: DictConfig) -> None:
    print(OmegaConf.to_yaml(cfg))
```

[…]

```
$ python train.py seed=2511 trainer.max_epochs=12
seed: 2511
trainer:
  accelerator: cpu
  max_epochs: 12
```

Hydra: создание объектов

```
.
├── config
│   └── train.yaml
└── train.py
```

```yaml
seed: 42
trainer:
  _target_: lightning.Trainer
  accelerator: cpu
  max_epochs: 10
```

```python
import hydra
from omegaconf import DictConfig, OmegaConf


@hydra.main(version_base=None, config_path="config", config_name="train")
def train(cfg: DictConfig) -> None:
    trainer = hydra.utils.instantiate(cfg.trainer)
    trainer.fit(...)
```

Hydra: частичная инициализация

```
.
├── config
│   └── train.yaml
└── train.py
```

```yaml
seed: 42
optimizer:
  _target_: torch.optim.Adam
  _partial_: true
  lr: 0.001
```

```python
import hydra
from omegaconf import DictConfig, OmegaConf


@hydra.main(version_base=None, config_path="config", config_name="train")
def train(cfg: DictConfig) -> None:
    model = ...
    optimizer_fn = hydra.utils.instantiate(cfg.optimizer)
    optimizer = optimizer_fn(model.parameters())
```

Hydra: композиция конфигурации

```
.
├── config
│   ├── model
│   │   ├── gru.yaml
│   │   └── lstm.yaml
│   └── train.yaml
└── train.py
```

config/train.yaml

```yaml
defaults:
  - model: gru


seed: 42
trainer:
  _target_: lightning.Trainer
  accelerator: cpu
  max_epochs: 10
```

config/model/gru.yaml

```yaml
model:
  _target_: torch.nn.GRU
  input_size: 8
  hidden_size: 8
```

config/model/lstm.yaml

```yaml
model:
  _target_: torch.nn.LSTM
  input_size: 8
  hidden_size: 8
```

```
$ python train.py model=lstm
```

Множественный запуск:

```
$ python train.py --multirun model=gru,lstm
```

MLFlow: трекинг экспериментов

Индустриальный стандарт

Ручное и автоматическое логирование

Визуализация метрик

Хранение артефактов

Реестр моделей

Версионирование датасетов

```python
import mlflow

mlflow.pytorch.autolog()  # That's it! 🎉

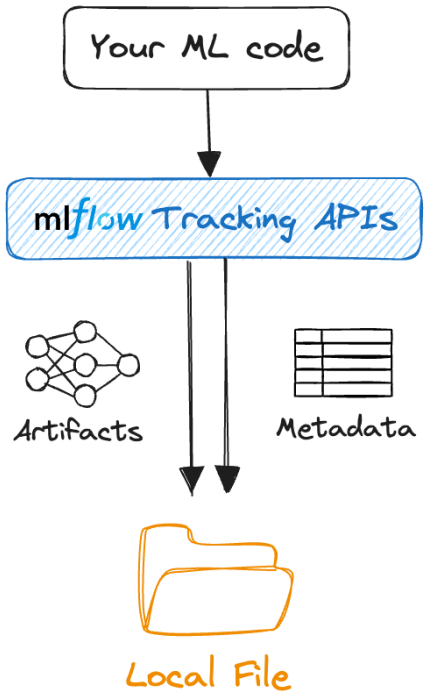
# Your existing PyTorch code works unchanged
for epoch in range(num_epochs):
    model.train()
    # ... your training loop stays exactly the same
```

Автоматическое логирование для Pytorch

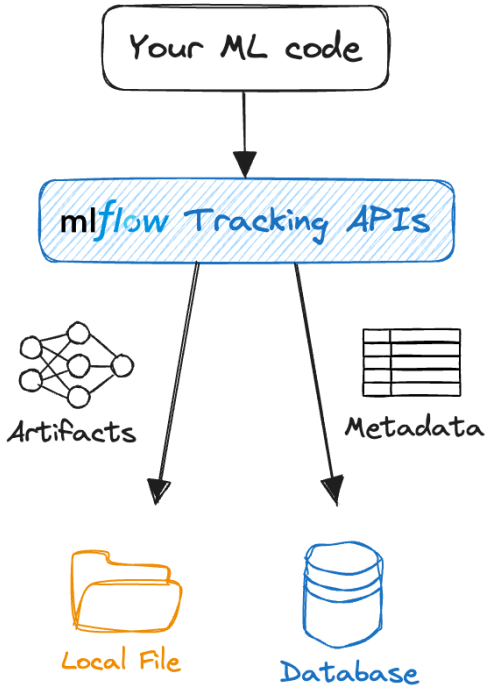
https://mlflow.org/docs/latest/ml/deep-learning/pytorch/

MLFlow: сценарии развёртывания

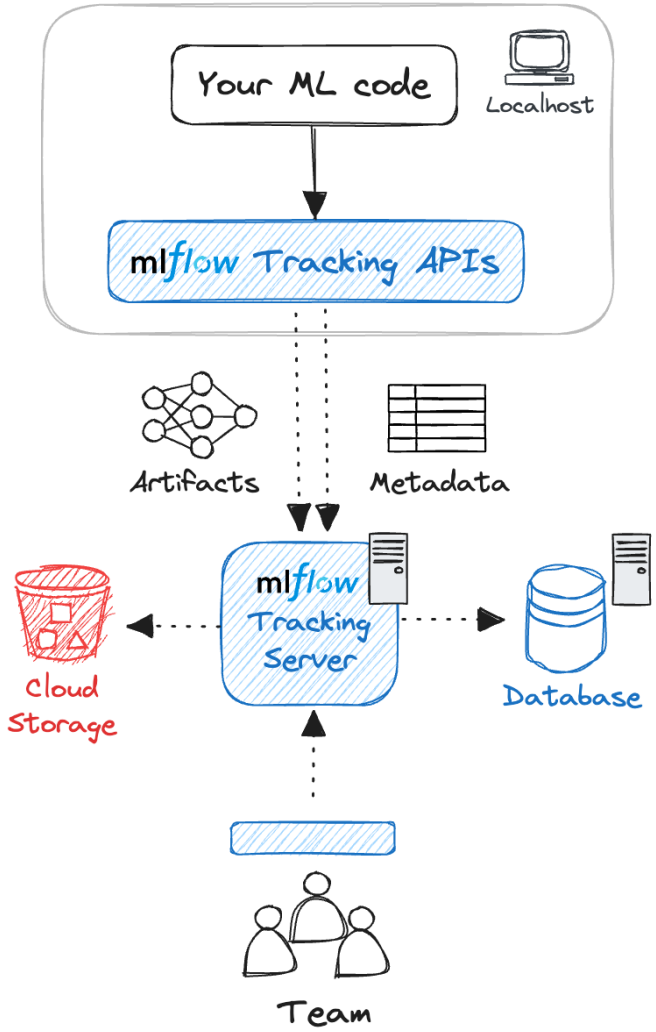
1. Localhost (default)



2. Localhost w/ various data stores



3. Remote Tracking w/ Tracking Server



DVC: версионирование данных



Инициализация проекта (внутри git репозитория)

```
$ dvc init
```

Добавление файлов

```
$ dvc add data/train.csv
```

1. Данные перемещаются в cache (может быть общим на несколько проектов)
2. В проект добавляется ссылка на объект в cache

Настройка удалённого хранилища

```
$ dvc remote add -d storage s3://mybucket/dvcstore
```

Загрузка в удалённое хранилище

```
$ dvc push
```

Скачивание

```
$ dvc pull
```

Переключение между версиями

```
$ git checkout <...>
$ dvc checkout
```

Ссылки

1. Hydra: https://hydra.cc/
2. MLFlow: https://mlflow.org/
3. Data Version Control: https://dvc.org/
4. Шаблон проекта с Lightning + Hydra: https://github.com/ashleve/lightning-hydra-template

Начало работы над проектом

Пререквизиты:

1. Проблема сформулирована, данные подготовлены
2. Пайплайн для обучения и тестирования настроен

Подробнее: Deep Learning Tuning Playbook

https://developers.google.com/machine-learning/guides/deep-learning-tuning-playbook/new-project

Старт проекта: архитектура

Постарайтесь найти документированную кодовую базу, решающую похожую задачу

Воспроизведите найденную модель в качестве отправной точки

Если такой нет — используйте популярную

Заставьте её работать, потом можно переключиться на собственные варианты

Старт проекта: оптимизатор

Нет “самого лучшего” оптимизатора, возможны вариации от задачи к задаче

Все параметры оптимизатора могут иметь значение, чем их больше — тем тщательнее должна быть настройка

Начните с простого оптимизатора с небольшим набором гиперпараметров: SGD с инерцией, Adam с фиксированными beta1 и beta2

На более поздних стадиях можно настроить все гиперпараметры

Старт проекта: размер мини-батча

Как правило, чем больше — тем лучше

Увеличение размера батча сокращает время обучения

Его не стоит трактовать как настраиваемый гиперпараметр: он не имеет значения при следующих условиях:

1. Параметры оптимизатора хорошо настроены

2. Регуляризация достаточна и откалибрована

3. Число шагов обучения достаточно

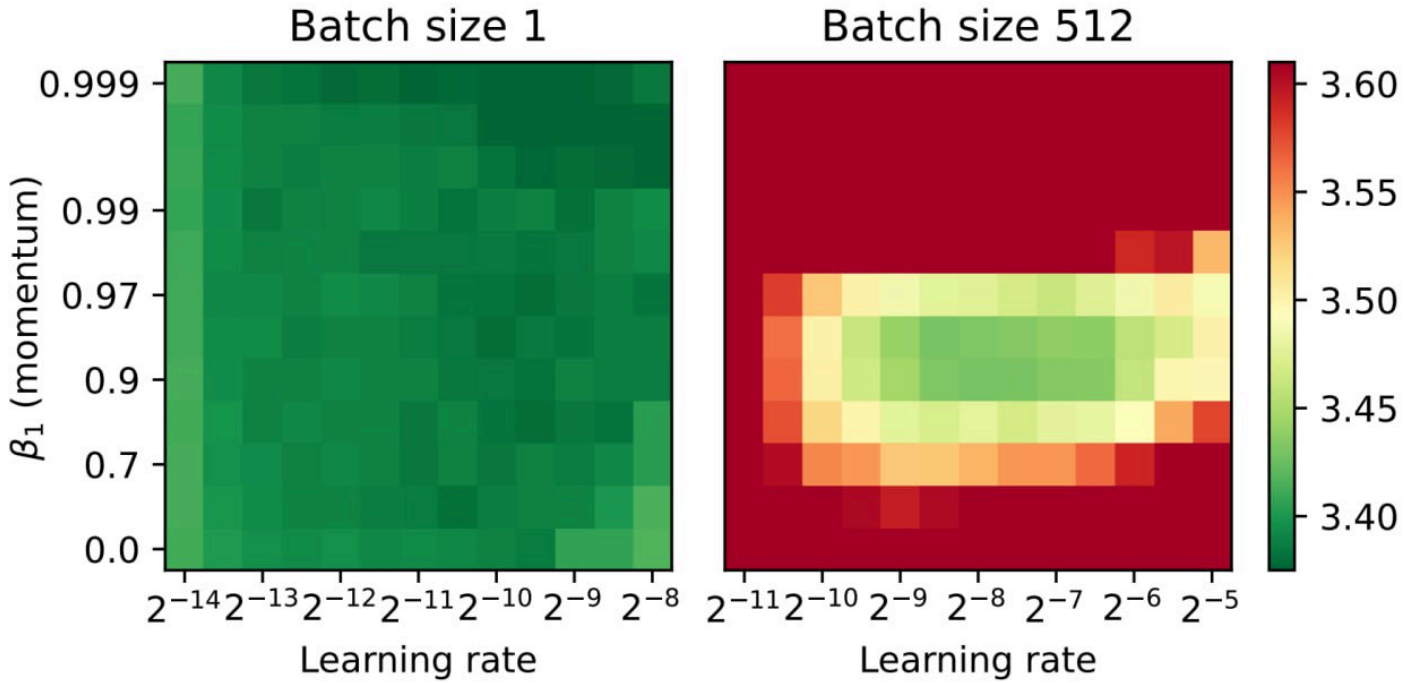
Изменение размера батча требует новой настройки большинства гиперпараметров (оптимизатор и регуляризация)

Shallue et al. (2018) Measuring the Effects of Data Parallelism on Neural Network Training

Старт проекта: размер мини-батча

Как правило, чем больше — тем лучше

Но есть нюанс:



(b) Small batches are more robust to hyperparameters

Marek et al. (2025) Small Batch Size Training for Language Models: When Vanilla SGD Works, and Why Gradient Accumulation Is Wasteful

Старт проекта: начальная конфигурация

Простая: без лишних наворотов вроде нетипичной архитектуры или регуляризации

Относительно быстрая и низкозатратная: проще настраивать гиперпараметры

Разумная точность: лучше случайного выбора

Баланс между достаточным числом шагов и скоростью итераций

Итеративное улучшение: стратегия

Начинаем с простого, постепенно вносим изменения, копим интуицию о задаче

Этапы:

1. Выберите цель очередного раунда экспериментов
2. Спроектируйте и запустите эксперименты
3. Проведите оценку результатов
4. Примите решение о вносимых изменениях

Определяем цель экспериментов

Примеры:

- Оценить потенциальное улучшение (новый регуляризатор, способ обработки данных)
- Оценить влияние определённого гиперпараметра
- Минимизировать ошибку валидации

Приоритет: новое понимание

Дизайн экспериментов: категории гиперпараметров

- исследуемые (scientific): их влияние хотим измерить
- мешающие (nuisance): их оптимизируем для исключения влияния
- фиксированные (fixed): их не меняем в раунде экспериментов

Роль гиперпараметра определяется целью эксперимента.

Пример для ReLU:

1. Что лучше: ReLU или tanh?
2. 5 или 6 слоёв для модели без ограничений на выбор функции активации?
3. Нужна ли батч-нормализация для ReLU-сетей?

Дизайн экспериментов: набор экспериментов

Каждый эксперимент — это

- набор варьируемых гиперпараметров и их возможные значения
- число испытаний
- алгоритм для семплирования конфигураций из пространства поиска

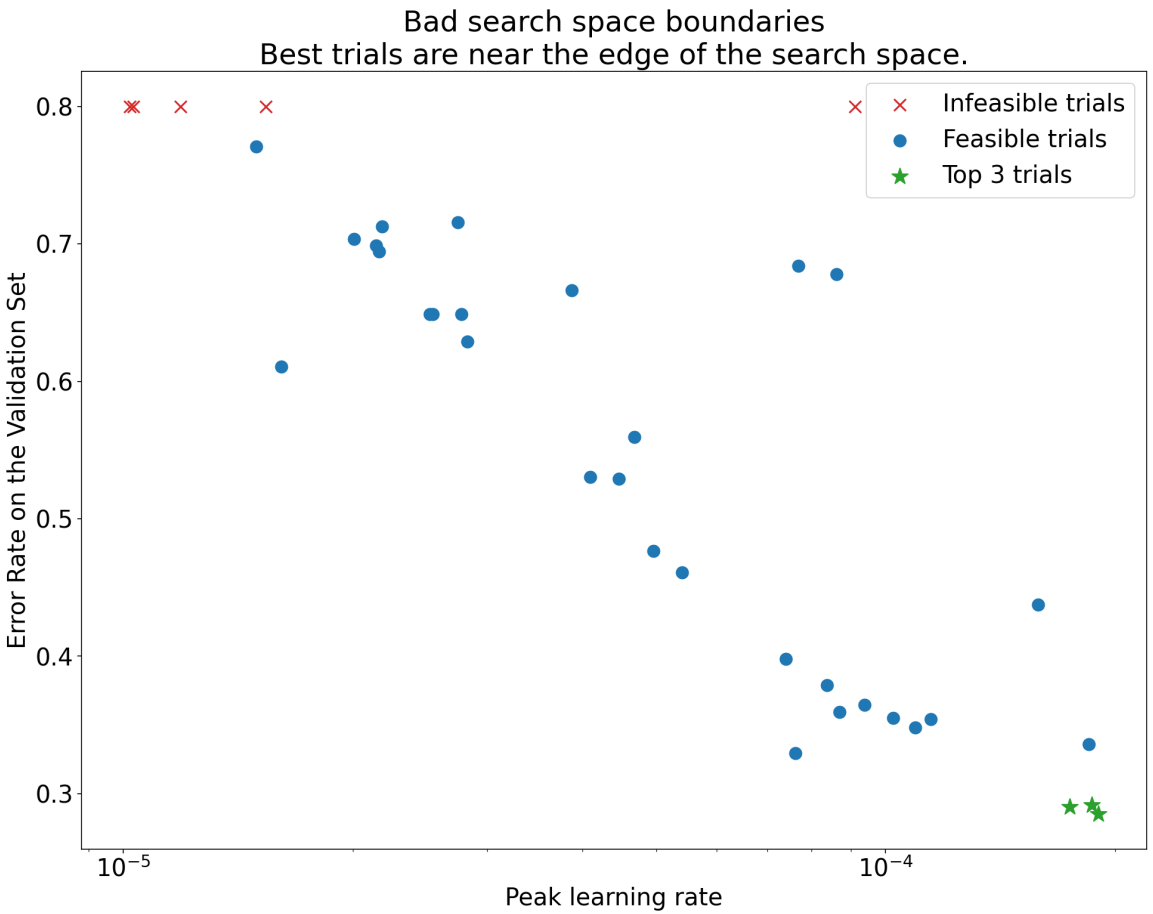
NB: Hydra поддерживает Optuna — фреймворк для оптимизации гиперпараметров: https://optuna.org/

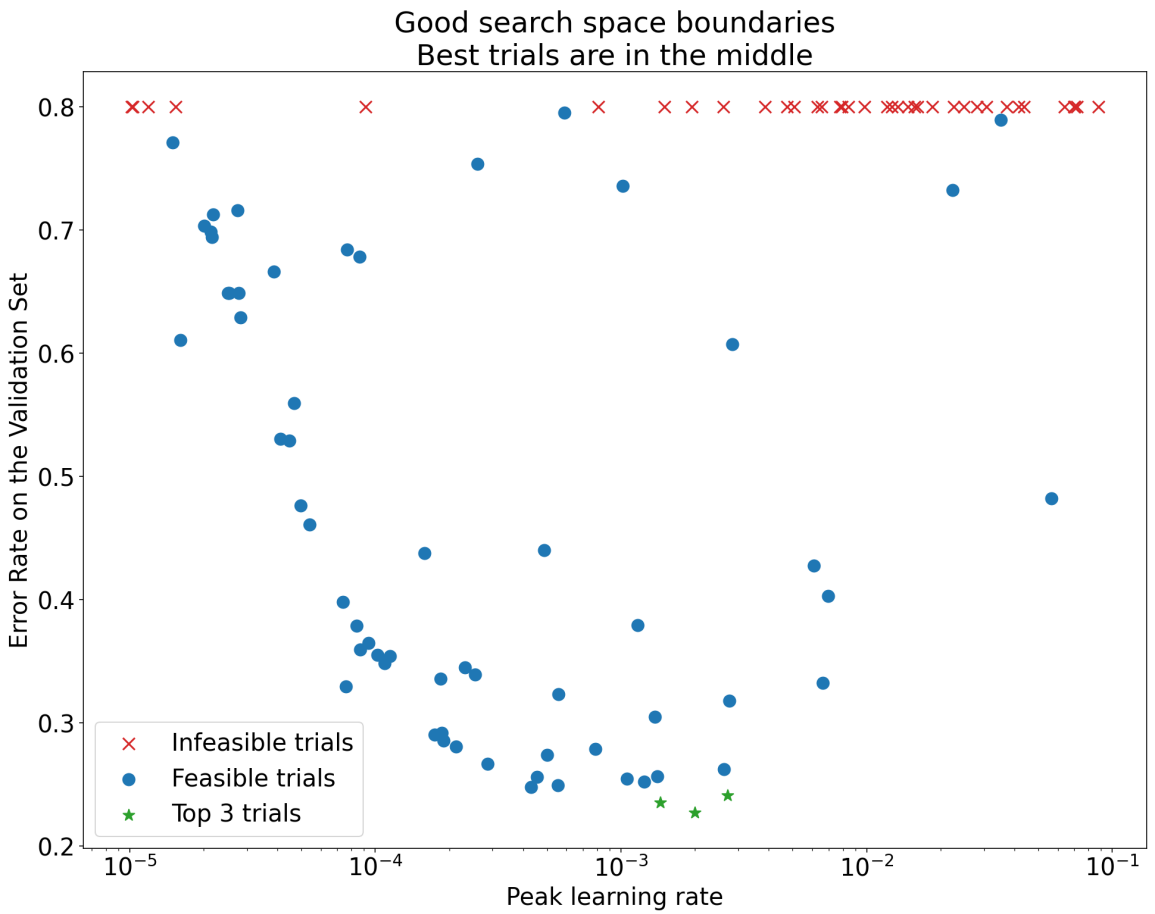
Оценка результатов

Чеклист:

- Достаточно ли пространство поиска?
- Достаточно ли испытаний проведено?
- Есть ли проблемы с обучением модели?

Оценка результатов: пространство поиска





Оценка результатов: причины проблем с обучением

Переобучение:

- добавляем регуляризацию
- донастраиваем имеющуюся

Высокая дисперсия от шага к шагу:

- увеличение размера батча
- увеличение валидационной выборки
- learning rate decay
- усреднение параметров модели

Принятие изменений

Следим за дисперсией результатов испытаний на нескольких запусках

Если новый вариант лучше текущего с учётом дисперсии между запусками — принимаем его за новый бейзлайн

Принимаем изменения, чей эффект перевешивает вносимую сложность